

CONCEPTOS DEL DÍA · SESIÓN 15

Control de versiones con git y GitHub

Documento de consulta para la persona estudiante

Módulo	CSTI12010 Diseño de algoritmos
Unidad de aprendizaje	UA2 · Programación estructurada en C
Capacidad	2.1 Emplear los principales elementos semánticos y sintácticos del lenguaje de desarrollo en el paradigma de programación estructurada.
Saberes	Elementos del ambiente de desarrollo: control de versiones · git y GitHub · repositorio · áreas de trabajo · commit · remoto · ramas y pull requests · portafolio del curso
Persona facilitadora	Giovanni Antonio Coto Calderón

Para qué le sirve este documento

Resume lo visto en la presentación. Téngalo a mano durante la UA2 y la UA3: al final encontrará la tabla de comandos, la rutina de cada sesión y la solución a los errores más comunes.

1. El problema de las versiones

Cuando se guardan copias con otro nombre (programa_final.c, programa_final2.c, programa_final2_AHORASI.c) nadie sabe con certeza cuál es la última versión, qué cambió entre una y otra, quién hizo cada cambio ni cómo volver a la versión que funcionaba. Con varias personas trabajando en el mismo proyecto, el problema se multiplica.

2. Sistema de control de versiones

Un **sistema de control de versiones** es una herramienta que **registra los cambios de los archivos de un proyecto a lo largo del tiempo**, de modo que se puede ver qué cambió, quién lo cambió, cuándo y por qué, y volver a cualquier versión anterior.

Beneficio	Qué significa
Historial completo	Cada cambio queda guardado con fecha, autor y descripción.
Volver atrás	Se recupera cualquier versión anterior en segundos.
Trabajo en equipo	Varias personas trabajan en el mismo proyecto sin sobrescribirse.
Respaldo	Una copia en la nube protege el trabajo si falla la computadora.

3. git y GitHub

	git	GitHub
¿Qué es?	Un programa de control de versiones.	Un servicio web que aloja repositorios git.
¿Dónde funciona?	En su computadora, incluso sin internet.	En la nube (github.com).
¿Para qué?	Registrar el historial del proyecto.	Respaldar, compartir y revisar código (pull requests).
Origen	Creado por Linus Torvalds en 2005; libre y gratuito.	Empresa fundada en 2008. Alternativas: GitLab, Bitbucket.

Analogía

git es la cámara que toma las fotos de su proyecto; **GitHub** es el álbum en la nube donde las guarda y las comparte. Se puede usar git sin GitHub, pero no GitHub sin git.

4. Repositorio

Un **repositorio** es la carpeta de un proyecto que git controla. Además de sus archivos, contiene una carpeta oculta **.git** donde se guarda todo el historial. **Nunca la borre ni la modifique a mano.**

- **Repositorio local:** la copia en su computadora; ahí trabaja y hace commits.
- **Repositorio remoto:** la copia en GitHub (se le llama **origin**); sirve de respaldo y de punto de entrega.

5. Las áreas de git

Un cambio recorre cuatro lugares antes de llegar a GitHub:



Figura 1. Recorrido de un cambio: del directorio de trabajo a GitHub.

Área	Qué contiene	Comando para pasar a la siguiente
Directorio de trabajo	Los archivos tal como los ve y edita.	git add <archivo>
Área de preparación (staging)	Los cambios elegidos para el próximo commit.	git commit -m "mensaje"
Repositorio local	El historial de commits, dentro de .git.	git push
Repositorio remoto	La copia del historial en GitHub.	—

git status muestra en qué área está cada cambio. Úselo siempre: no modifica nada.

6. El commit

Un **commit** guarda en el historial el estado de los archivos preparados. Es como una foto del proyecto en un momento dado. Cada commit tiene:

Salida real de git log -1

```
commit 3dd4e232574aa37d1862ea23483de1de7ccc5dcb    <- hash (identificador único)
Author: Ana Mora Solís <ana.mora@correo.com>      <- persona autora
Date:   Mon Sep 28 10:30:24 2026 -0600            <- fecha y hora

    UA1: agrega evidencias de la sesion 1          <- mensaje
```

El hash suele mostrarse abreviado a 7 caracteres (3dd4e23). El nombre y el correo salen de **git config**: por eso cada commit es su firma sobre el trabajo.

Mensajes de commit

Convención del curso: **SXX: verbo en presente + qué se hizo.**

Correcto	Incorrecto
S15: agrega programa hola.c	cambios
S16: corrige el cálculo del promedio	asdf
A2: agrega función para validar la edad	ya quedó (ahora sí)

7. Trabajar con el remoto

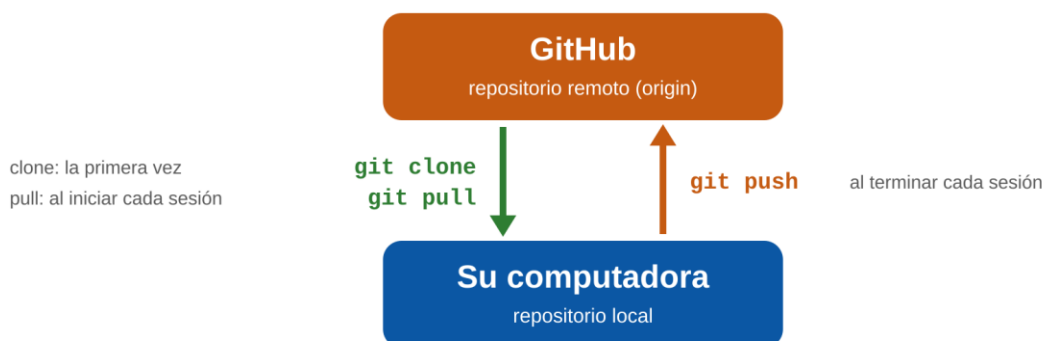


Figura 2. clone y pull traen cambios de GitHub; push los sube.

Comando	Cuándo se usa	Qué hace
git clone <url>	Una sola vez, al empezar	Descarga el repositorio completo con todo su historial.
git pull	Al iniciar cada sesión	Trae y combina los cambios nuevos de GitHub.
git push	Al terminar cada sesión	Sube sus commits a GitHub.

Comando	Cuándo se usa	Qué hace
Importante Un commit sin push solo existe en su computadora . Si no hace push, la persona docente no ve su trabajo y usted no tiene respaldo.		

8. Ramas y pull requests

Una **rama** es una línea de trabajo paralela a la principal (**main**). Un **pull request** es la solicitud para unir una rama a main; permite revisar los cambios y comentarlos línea por línea antes de unirlos. Una **etiqueta** (tag) marca una versión específica, por ejemplo la versión evaluada de una actividad.



Figura 3. La Actividad 2 se desarrolla en su rama, se entrega con un pull request y la versión evaluada queda etiquetada.

En el curso las usaremos para entregar las actividades de comprobación 2 y 3.

9. El portafolio del curso en GitHub

- La **organización del curso** es propiedad de la persona facilitadora.
- Cada estudiante tiene un **repositorio privado**: CSTI12010-2026-nombre-apellido. Solo usted y la persona docente tienen acceso.
- La estructura (UA1, UA2, UA3, actividades, docs) y el archivo **.gitignore** ya vienen en la plantilla.
- Las reglas de nombres, commits, ramas y entregas están en **docs/CONVENCIONES.md** dentro de su repositorio.
- Al terminar el módulo puede llevarse su portafolio a su cuenta personal.

.gitignore

Indica qué archivos no debe guardar git. En la plantilla se ignoran los ejecutables (.exe) y el código objeto (.o), porque se generan de nuevo al compilar. Con **git status --ignored --short** los archivos ignorados aparecen marcados con **!!**.

10. La rutina de cada sesión

Git Bash

```
git pull # 1. traer lo más reciente
# 2. trabajar en la carpeta de la sesión y completar su README.md
git status # 3. revisar qué cambió
git add UA2/S16 # 4. preparar la carpeta de la sesión
git commit -m "S16: agrega letrero.c y su evidencia"
git push # 5. subir a GitHub
```

11. Comandos esenciales

Comando	Para qué sirve
git clone <url>	Descargar un repositorio de GitHub (la primera vez).
git config user.name "Nombre"	Configurar su nombre en este repositorio.
git config user.email "correo"	Configurar su correo en este repositorio.
git status	Ver el estado de los archivos.
git add <archivo>	Pasar cambios al área de preparación.
git commit -m "mensaje"	Registrar los cambios con un mensaje.
git log --oneline	Ver el historial resumido.
git diff	Ver las líneas que cambiaron y aún no se preparan.
git push	Subir los commits a GitHub.
git pull	Bajar los cambios de GitHub.
git show <hash>:<archivo>	Ver cómo era un archivo en un commit anterior.
git restore <archivo>	Descartar los cambios no guardados en commit de un archivo.

12. Errores frecuentes y cómo resolverlos

Mensaje	Causa	Solución
fatal: not a git repository	La terminal no está dentro de la carpeta del repositorio.	Entre a la carpeta con cd y verifique con ls.
Author identity unknown	No configuró su nombre y correo.	git config user.name "..." y git config user.email "..."
nothing to commit, working tree clean	No hay cambios nuevos, o ya los registró.	Nada que hacer; revise con git log.
! [rejected] ... (fetch first)	GitHub tiene commits que usted no tiene.	git pull y luego git push.
remote: Repository not found	No ha aceptado la invitación o la dirección está mal escrita.	Acepte la invitación en GitHub y copie la dirección desde el botón Code.

Glosario

Término	Definición breve
Branch (rama)	Línea de trabajo paralela dentro de un repositorio.
Clone	Copia local completa de un repositorio remoto.
Commit	Registro de un conjunto de cambios, con autor, fecha y mensaje.
Hash	Código único que identifica cada commit.
main	Nombre de la rama principal.
origin	Nombre que git le da al repositorio remoto del que se clonó.
Pull request	Solicitud para unir una rama a otra, con revisión.
Push / Pull	Subir / bajar commits entre el repositorio local y el remoto.
Staging	Área de preparación: los cambios que irán en el próximo commit.
Tag (etiqueta)	Nombre fijo que marca un commit específico, como una versión entregada.

Preguntas de repaso

1. ¿Qué diferencia hay entre git y GitHub?
2. Describa el recorrido de un cambio desde que edita un archivo hasta que llega a GitHub.
3. ¿Qué información guarda cada commit?
4. ¿Qué pasa si hace commit pero no hace push?
5. ¿Por qué los archivos .exe no se suben al repositorio?
6. ¿Para qué se usarán las ramas y los pull requests en este curso?

Fuentes recomendadas

- Chacon, S. y Straub, B. Pro Git (2.ª ed.). Disponible en español y gratis en git-scm.com/book/es/v2.
- GitHub Docs. Documentación oficial de GitHub, disponible en español en docs.github.com/es.